# Token Efficient Language

Diseño e Implementación de un Lenguaje

## 1. Equipo


| Nombre | Apellido | Legajo | E-mail |
|---|---|---|---|
| Bautista | Pessagno | 65101 | bpessagno@itba.edu.ar |
| Rodrigo Alejandro | Hernandez | 65522 | rohernandez@itba.edu.ar |
| Lorenzo Alejandro | Méndez | 65147 | lmendez@itba.edu.ar |
| Pascal | Ordano | 65485 | pordano@itba.edu.ar |


## 2. Repositorio

La solución será versionada en: https://github.com/BautistaPessagno/TEL

# 3. Dominio

Desarrollar un DSL denominado TEL (Token-Efficient Language), cuyo objetivo es representar programas escritos en un subconjunto del lenguaje C utilizando una sintaxis compacta que minimiza la cantidad de tokens consumidos por Modelos de Lenguaje de Gran Escala (LLMs), pero que al mismo tiempo sea legible y entendible por un desarrollador. El compilador tomará como entrada un programa en TEL y producirá como artefacto de salida su equivalente en código C estándar, funcionando como un compilador fuente-a-fuente (*source-to-source compiler*).

El dominio es la optimización de tokens en la interacción humano-LLM para el desarrollo de software en C. Actualmente, cuando desarrolladores utilizan LLMs (como Claude, GPT, Gemini, etc.) para escribir, revisar o modificar código C, la verbosidad inherente del lenguaje C genera un consumo excesivo de tokens tanto en la entrada (prompts, lectura de archivos, output de comandos, etc) como en la salida (respuestas). Esto tiene un impacto directo en costos económicos (los proveedores de LLMs cobran por token), en latencia (más tokens implican mayor tiempo de respuesta), y en la ventana de contexto disponible (los tokens desperdiciados en sintaxis reducen el espacio para contenido semántico).

TEL aborda esta necesidad arquitectural definiendo una representación alternativa y más compacta de C que preserva la totalidad de la información semántica del programa original. La transformación es biyectiva: dado cualquier programa válido en TEL, el compilador produce un único programa C equivalente y bien formado, y a su vez, cualquier programa en el subconjunto de C soportado tiene una representación única en TEL. Para el alcance de este proyecto, solo se implementará la dirección TEL → C.

El subconjunto de C cubierto por TEL incluye: tipos primitivos (int, float, double, char, void), calificadores de tipo (unsigned, long, short, const), variables, arreglos, punteros básicos, funciones, estructuras de control (if/else, for, while, do-while, switch), structs, enums, typedef, punteros a función, y directivas de preprocesador básicas (#include, #define).

La implementación satisfactoria de este lenguaje permitiría reducir significativamente el costo y la latencia de las interacciones con LLMs en flujos de trabajo de desarrollo en C, habilitando a equipos de ingeniería a procesar más código dentro de una misma ventana de contexto y obtener respuestas más rápidas, sin perder información semántica alguna.

# 4. Construcciones

El lenguaje TEL ofrece las siguientes construcciones, prestaciones y funcionalidades. Es importante aclarar que muchas palabras reservadas no se verán afectadas ya que los LLMs han logrado reducir dichas palabras a un solo token:

(I). Keywords de tipo compactos: Se reemplazan los tipos de C por variantes cortas que ocupan un único token. Por ejemplo: int → int, char → char, float → float, double → double, uint → unsigned int, long → long, v → void, uli -> unsigned long int. Esto reduce múltiples tokens de tipo compuesto (unsigned long int = 3 tokens) a uno solo (uli = 1 token).

(II). Declaración de funciones compacta: Se utiliza la keyword fn en lugar de colocar el tipo de retorno antes del nombre. La sintaxis es fn nombre params -> tipo_retorno. Si la función retorna void, se omite la flecha y el tipo de retorno. Se utiliza ret en lugar de return, o si la última línea es una variable o valor del tipo de retorno de la funcion, se agrega un return implicito.

(III). Eliminación de punto y coma: Las sentencias se terminan con salto de línea en lugar de ;. Esto ahorra un token por cada sentencia del programa. Para múltiples sentencias en una línea, se puede usar explícitamente como separador.

(IV). Bloques por indentación: Los bloques de código se delimitan por indentación (similar a Python) en lugar de { }. Esto elimina dos tokens (apertura y cierre de llave) por cada bloque.

(V). Estructuras de control compactas: Se proveen keywords cortos para control de flujo: ? → if, elif → else if, else → else,

luego para las condiciones se va tomar en cuenta ciertos defaults usados frecuentemente en la programación. por ejemplo el for(int i=0; i < n; i++) será reemplazado por el fd i 0 n. Luego se aplicaran los generales for → for, w → while, dw → do-while, sw → switch, num: → case, df → default, brk → break, cnt → continue.

(VI). Operadores aritméticos y relacionales: Se mantienen los operadores estándar de C: +, -, *, /, %, ==, !=, <, >, <=, >=, así como los operadores de asignación compuesta (+=, -=, etc.) y los operadores de incremento/decremento (++, --).

(VII). Operadores lógicos y bit a bit: Se proveen: &&, ||, ! para lógica booleana, y &, |, ^, ~, <<, >> para operaciones bit a bit, manteniendo compatibilidad directa con la semántica de C.

(VIII). Directivas de preprocesador simplificadas:#include → #include, #define → #define. Además, los includes de librerías estándar se simplifican: #include stdio → #include <stdio.h> (se infiere la extensión .h automáticamente para headers del sistema).

(IX). Structs, enums y unions compactos: Se utilizan keywords reducidos: str → struct, en → enum, td → typedef. Los bloques de definición de structs también utilizan indentación en lugar de llaves.

(X). Punteros y arreglos: Se soportan punteros básicos con la sintaxis estándar * y arreglos con []. Los punteros a función se declaran con una sintaxis simplificada: fn* nombre tipos -> tipo en lugar de tipo (*nombre)(tipos).

(XI). Variables y constantes: Se pueden declarar variables con su tipo explícito (x:int = 5). Se soporta la declaración static con el keyword stc.

(XII). Literales: Se soportan literales enteros (decimal, hexadecimal con 0x, octal con 0o), de punto flotante, de caracter ('a'), strings ("hola"), y el literal null → NULL.

(XIII). Comentarios: Se mantienen los comentarios de línea con // y los comentarios de bloque con /* */, preservando su contenido intacto en la salida C generada.

(XIV). Entry point: Todo programa TEL debe contener una función main como punto de entrada, la cual se traduce a int main(int argc, char* argv[]) en C.

(XV). I/O implícito: Si el programa utiliza funciones de I/O estándar (printf, scanf, puts, etc.), el compilador incluye automáticamente #include <stdio.h> en la salida sin necesidad de que el programador lo especifique en TEL. Análogamente para malloc/free se incluye #include <stdlib.h> y para strlen/strcpy se incluye #include <string.h>.

# 5. Casos de prueba

Se proponen los siguientes casos iniciales de prueba de aceptación:
(I). Un programa TEL mínimo con un main que imprime un string, que se traduce correctamente a C con el include de stdio.h generado automáticamente.

(II). Un programa TEL que declara y utiliza un arreglo de enteros con un bucle for, que se traduce a un for clásico de C con llaves y punto y coma.

(III). Un programa TEL que define una función con parámetros y tipo de retorno usando la sintaxis fn/ret, que se traduce correctamente a una función C con la firma expandida.

(IV). Un programa TEL que utiliza estructuras de control compactas (for, elif, else, w) con bloques por indentación, que se traduce a if/else if/else/while con llaves en C.

(V). Un programa TEL que define un struct con la keyword str y campos tipados, que se traduce a una definición struct completa en C con llaves y punto y coma.

(VI). Un programa TEL que utiliza typedef (td), struct (str) y literales diversos (hexadecimales, strings), que se traduce preservando todos los calificadores en C.

(VII). Un programa TEL que utiliza directivas de preprocesador compactas (#include, #define), que se traduce a las directivas expandidas (#include con extensión .h, #define).

(VIII). Un programa TEL que utiliza punteros, incluyendo un puntero a función con la sintaxis fn*, que se traduce a la declaración C equivalente con la notación estándar de punteros a función.

(IX). Un programa TEL que combina múltiples construcciones (enum, switch con sw/df/brk, funciones, structs) en un programa cohesivo, demostrando la interacción entre las features.

(X). Un resolución de un problema conocido de C (como N-QUEENS) en TEL para demostrar la capacidad de un LLMs para interactuar con el lenguaje, que se traduce a un código equivalente funcional en C.

Además, los siguientes casos de prueba de rechazo:
(I). Un programa TEL con indentación inconsistente (mezcla de tabs y espacios o niveles de indentación incorrectos), que resulta en un error de parseo.

(II). Un programa TEL que utiliza un keyword reservado como identificador de variable (por ejemplo, nombrar una variable fn, ret, str, o sw).

(III). Un programa TEL con una expresión aritmética mal formada (como operadores faltantes)

(IV). Un programa TEL que define una función con la sintaxis de tipo de retorno inválida (por ejemplo, omitir el tipo después de la flecha ->).

(V). Un programa TEL con un literal string sin cerrar (comillas de apertura sin comillas de cierre).

# 6. Ejemplos

El ejemplo consiste del código C (izquierda) comparado con el lenguaje TEL (derecha) con sus respectivos conteo de tokens (usando https://platform.openai.com/tokenizer )

## 6.1 Un simple hola mundo

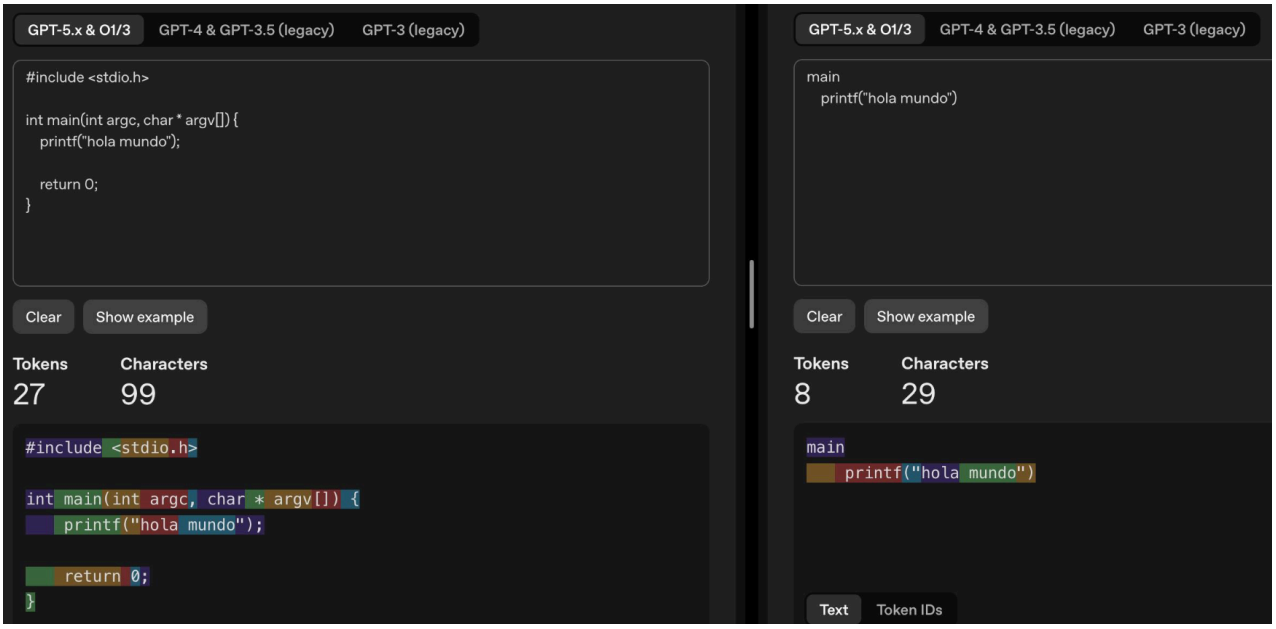


Figura 1: Diferencia de tokens para programa hola mundo

## 6.2 Una definición de una función la cual usa un for default para recorrer un array

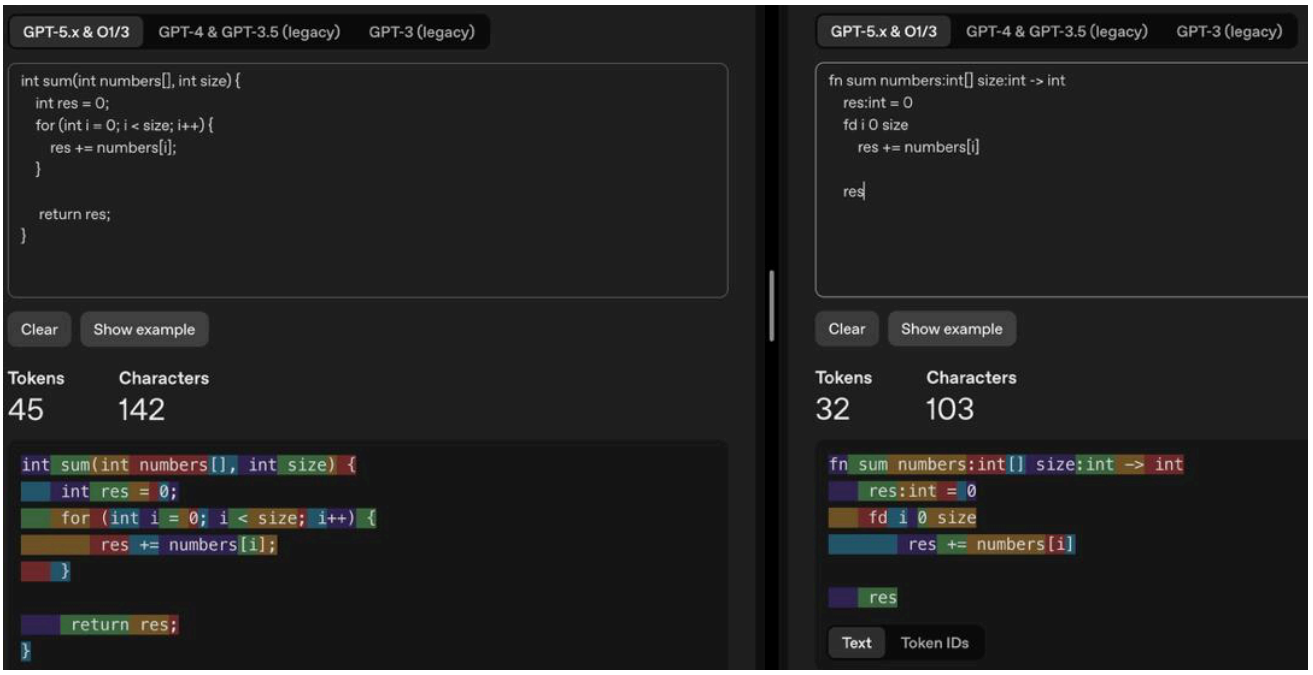


Figura 2: Diferencia de tokens para un bucle for

## 6.3 Definición de un struct

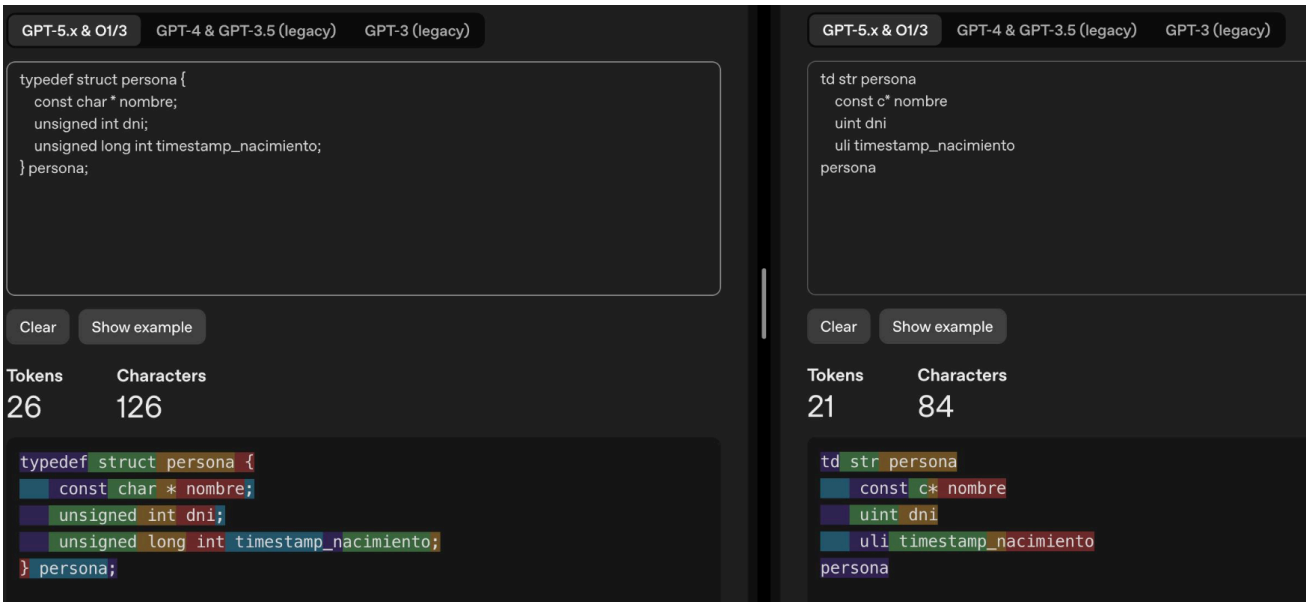


Figura 3: Diferencia de tokens para la definición de un struct

## 6.4 uso de #Includes

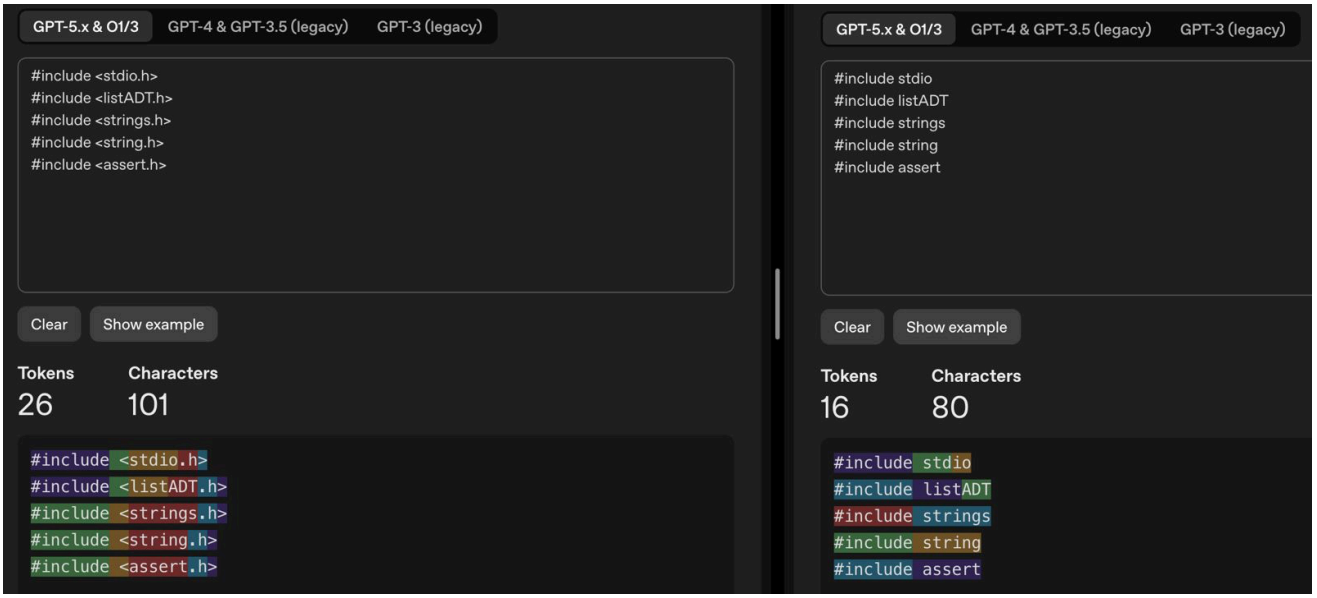


Figura 3: Diferencia de tokens para el uso de includes